Regression
(numerical prediction)

Regression

- A supervised learning method
- Models continuous-valued functions, i.e., predicts unknown numeric values
- Analyze and understand relationships among several quantities
- Build a model that that predicts the value of a variable (dependent variable) as a function of other variables (independent variables)

Linear Regression / least-squares line

- The simplest relation between two variables x and y is the linear equation:

$$(x_1, y_1), \ldots, (x_n, y_n)$$

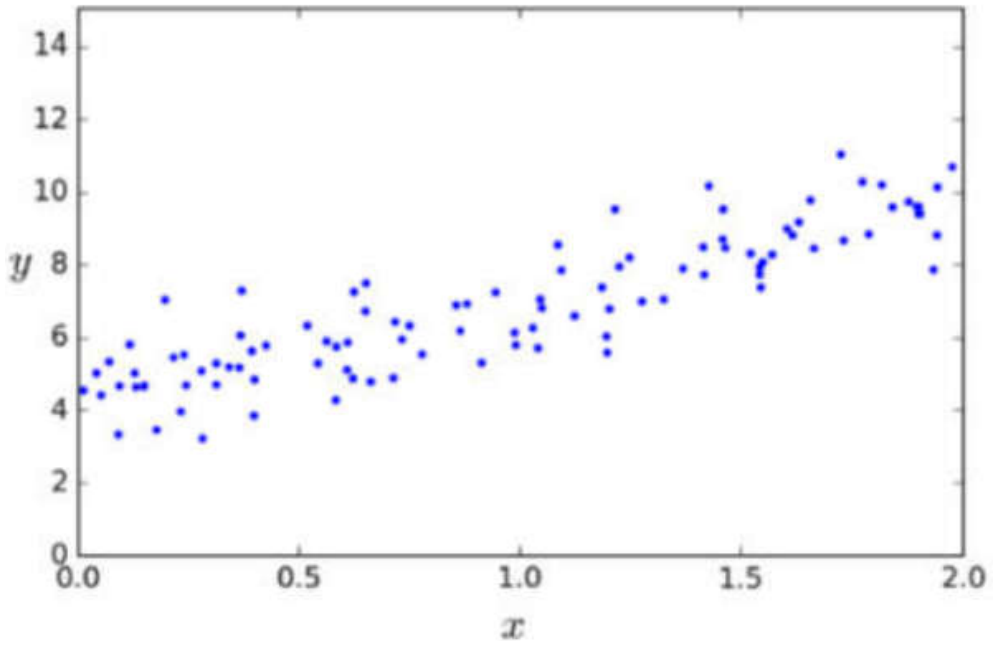
$$y = \beta_0 + \beta_1 x$$

  - The above notation is commonly used for least squares lines instead of $y = mx + b$

Linear Regression (cont.)

$$(x_1, y_1), \ldots, (x_n, y_n)$$

$$y = \beta_0 + \beta_1 x$$



Linear Regression (cont.)

- If the data points were on a line, the parameters would satisfy the equations:

Predicted y-value		Observed y-value
$\beta_0 + \beta_1 x_1$	=	y_1
$\beta_0 + \beta_1 x_2$	=	y_2
$\vdots$		$\vdots$
$\beta_0 + \beta_1 x_n$	=	y_n

Linear Regression (cont.)

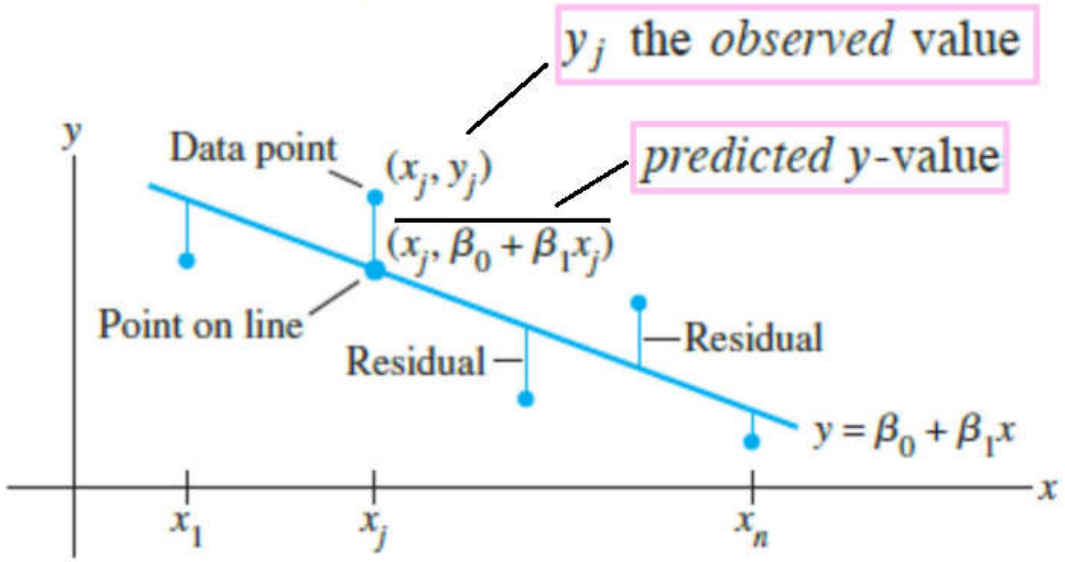
Predicted y-value		Observed y-value
$\beta_0 + \beta_1 x_1$	=	y_1
$\beta_0 + \beta_1 x_2$	=	y_2
$\vdots$		$\vdots$
$\beta_0 + \beta_1 x_n$	=	y_n

$$X = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix}, \quad \beta = \begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

$$y = X\beta + \epsilon$$

Linear Regression (cont.)

- If the data points do not lie on a line, then the are no parameters β_0, β_1 for which the predicted y-values in $X\beta$ equal the observed y-values.
  - Solution? Least squares line / line of regression



Linear Regression (cont.)

- There are several ways to measure how close the line is to the data.

- The usual choice is to add the squares of the residuals. The least-squares line is the line $y = \beta_0 + \beta_1 x$, that minimizes the sum of the squares of the residuals.

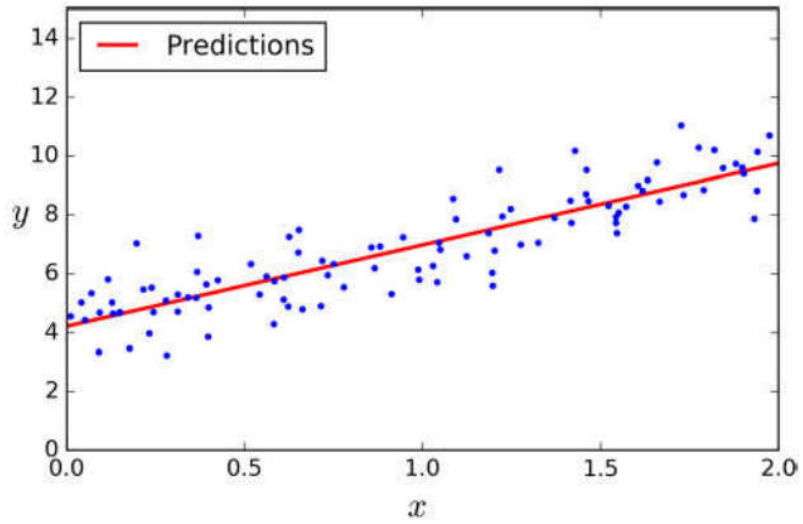
$$residual = \epsilon = y - X\beta$$

Cost Function

- Cost function = the sum of (squares of) the residuals

$$\epsilon^2 = (y - X\beta)^2$$

- The goal is to minimize the cost function



Cost Function (cont.)

$$\left|\left|\epsilon^2\right|\right| = (y - X\beta)^T(y - X\beta) = (X\beta)^T X\beta - (X\beta)^T y - y^T X\beta + y^T y$$

$$= (X\beta)^T X\beta - 2(X\beta)^T y + y^T y$$

$$\frac{\partial\left|\left|\epsilon^2\right|\right|}{\partial\beta} = 2X^T X\beta - 2X^T y = 0 \rightarrow \beta = (X^T X)^{-1} X^T y$$

Example

- Find the equation $y = \beta_0 + \beta_1 x$ of the least-squares line that best fits the data points $(2, 1), (5, 2), (7, 3)$, and $(8, 3)$.

- **Solution**:

$$X = \begin{bmatrix} 1 & 2 \\ 1 & 5 \\ 1 & 7 \\ 1 & 8 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 3 \end{bmatrix}$$

- For the least-squares solution of $X\boldsymbol{\beta} = \boldsymbol{y}$, obtain the following normal equation.

$$X^T X\boldsymbol{\beta} = X^T \boldsymbol{y}$$

Solution (cont.)

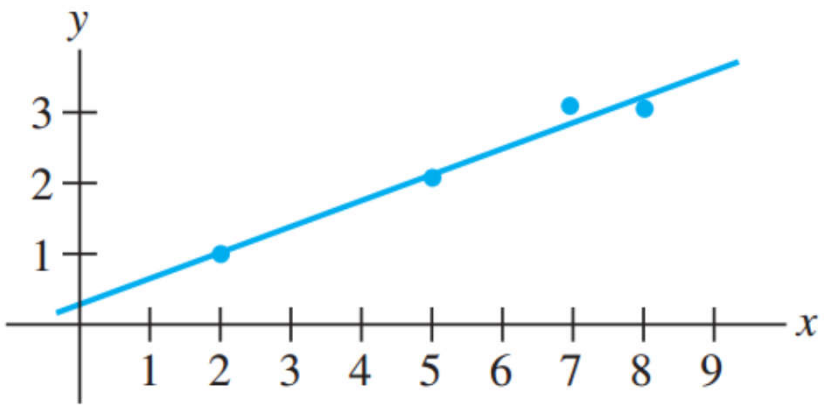
$$X^T X = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & 5 & 7 & 8 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 1 & 5 \\ 1 & 7 \\ 1 & 8 \end{bmatrix} = \begin{bmatrix} 4 & 22 \\ 22 & 142 \end{bmatrix}$$

$$X^T \mathbf{y} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & 5 & 7 & 8 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \\ 3 \end{bmatrix} = \begin{bmatrix} 9 \\ 57 \end{bmatrix}$$

Solution (cont.)

$$\begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix} = \begin{bmatrix} 4 & 22 \\ 22 & 142 \end{bmatrix}^{-1} \begin{bmatrix} 9 \\ 57 \end{bmatrix} = \frac{1}{84} \begin{bmatrix} 142 & -22 \\ -22 & 4 \end{bmatrix} \begin{bmatrix} 9 \\ 57 \end{bmatrix} = \frac{1}{84} \begin{bmatrix} 24 \\ 30 \end{bmatrix} = \begin{bmatrix} 2/7 \\ 5/14 \end{bmatrix}$$

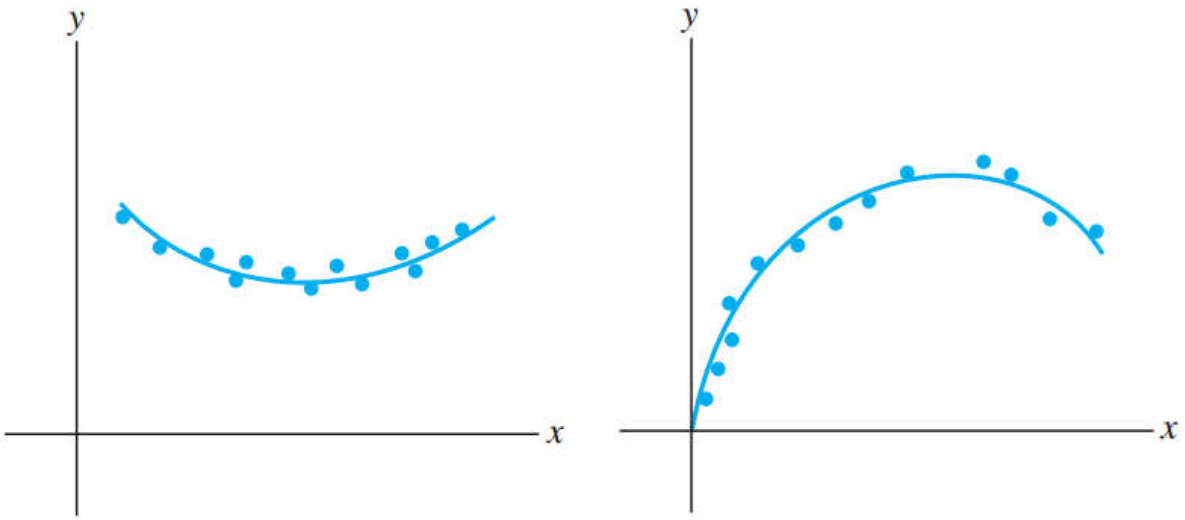
$$y = \frac{2}{7} + \frac{5}{14}x$$



Non-linear Regression / Curve fitting

- When data points $(x_1, y_1), \ldots, (x_n, y_n)$ on a scatter plot do not lie close to any line, some other functional relationship between x and y may be tried:

$$y = \beta_0 f_0(x) + \beta_1 f_1(x) + \cdots + \beta_k f_k(x)$$



Example

- Suppose data points appear to lie along some sort of parabola. More precisely, we wish to approximate the data by an equation of the form:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2$$

Describe the model that produces a least-squares fit of the data.

Solution

$$
\begin{aligned}
y_1 &= \beta_0 + \beta_1 x_1 + \beta_2 x_1^2 + \epsilon_1 \\
y_2 &= \beta_0 + \beta_1 x_2 + \beta_2 x_2^2 + \epsilon_2 \\
&\vdots \qquad\qquad\qquad \vdots \\
y_n &= \beta_0 + \beta_1 x_n + \beta_2 x_n^2 + \epsilon_n
\end{aligned}
$$

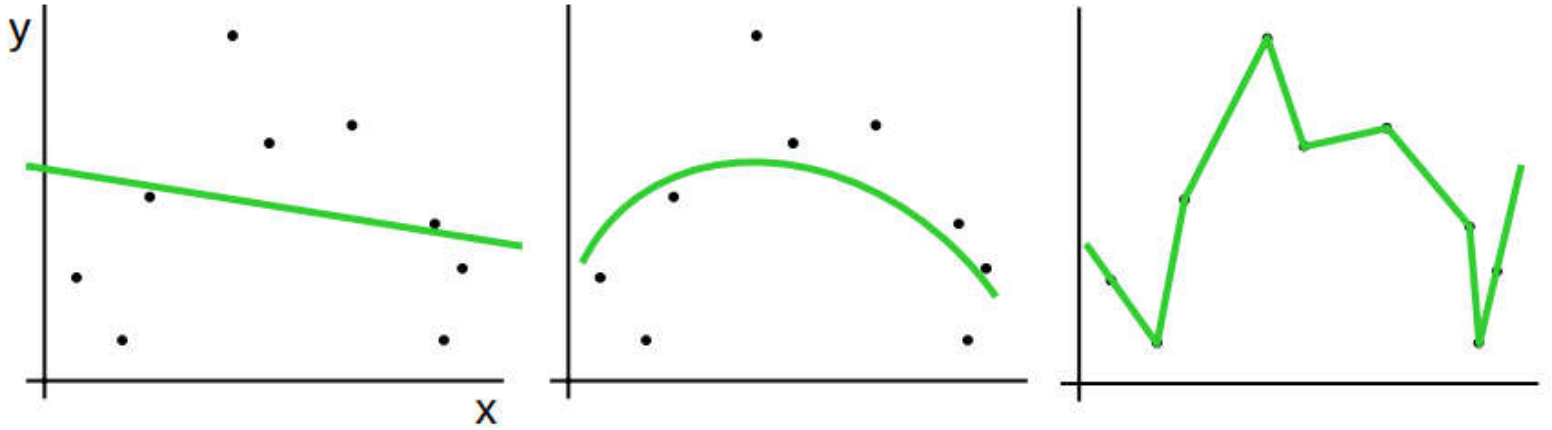
where ϵ_i is the residual error between the observed value y_i and the predicted y-value.

Solution (cont.)

$$\mathbf{y} = X \boldsymbol{\beta} + \boldsymbol{\epsilon}$$

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix} + \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{bmatrix}$$

Overfitting vs. Underfitting



$y = \beta_0 + \beta_1 x$ $y = \beta_0 + \beta_1 x + \beta_2 x^2$ $y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \cdots$

Multiple Regression

- Suppose we have two independent variables, say u and v, and we wish to predict y (dependent variable). A simple (linear) equation for predicting y from u and v:

$$y = \beta_0 + \beta_1 u + \beta_2 v$$

- A more general prediction equation:

$$y = \beta_0 + \beta_1 u + \beta_2 v + \beta_3 u^2 + \beta_4 uv + \beta_5 v^2$$

Multiple Regression (cont.)

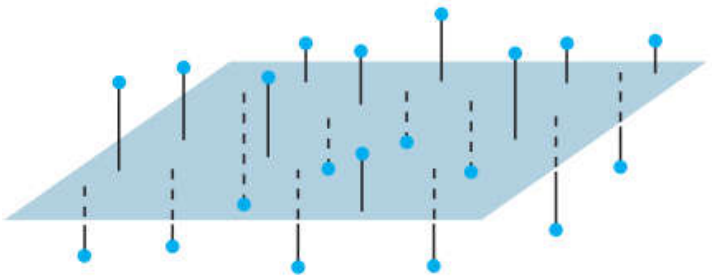
- In general, a linear model will arise whenever y is to be predicted by an equation of the form:

$$y = \beta_0 f_0(u, v) + \beta_1 f_1(u, v) + \cdots + \beta_k f_k(u, v)$$

with $f_0, \ldots, f_k$ any sort of known functions & $\beta_0, \ldots, \beta_k$ unknown parameters.

Example

- Suppose we have training data of the form: $(u_1, v_1, y_1), \ldots, (u_n, v_n, y_n)$
- Describe the linear model that gives a least-square fit to such data. The solution is called the least-squares plane.



Solution

- We expect the data to satisfy the following equations:

$$\begin{aligned} y_1 &= \beta_0 + \beta_1 u_1 + \beta_2 v_1 + \epsilon_1 \\ y_2 &= \beta_0 + \beta_1 u_2 + \beta_2 v_2 + \epsilon_2 \\ &\vdots \qquad\qquad \vdots \\ y_n &= \beta_0 + \beta_1 u_n + \beta_2 v_n + \epsilon_n \end{aligned}$$

- The system has the matrix form $\boldsymbol{y} = X\boldsymbol{\beta} + \boldsymbol{\epsilon}$, where

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}, \quad X = \begin{bmatrix} 1 & u_1 & v_1 \\ 1 & u_2 & v_2 \\ \vdots & \vdots & \vdots \\ 1 & u_n & v_n \end{bmatrix}, \quad \boldsymbol{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix}, \quad \boldsymbol{\epsilon} = \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{bmatrix}$$

Solution (cont.)

- This example shows that the linear model for multiple regression has the same abstract form as the model for the simple regression in the earlier examples.

- Once X is defined properly, the normal equations for β have the same matrix form, no matter how many variables are involved.

➢For any linear model where $X^T X$ is invertible, the least squares β is given by $(X^T X)^{-1} X^T \boldsymbol{y}$.

Gradient Descent (2nd Approach)

- Linear Regression Hypothesis space:

$$h(x) = \theta_0 + \theta_1 x$$

  - Goal is to learn parameters θ_0, θ_1

How to Learn Parameters

- Parameters θ_0 and θ_1 are estimated by minimizing a cost/loss function:

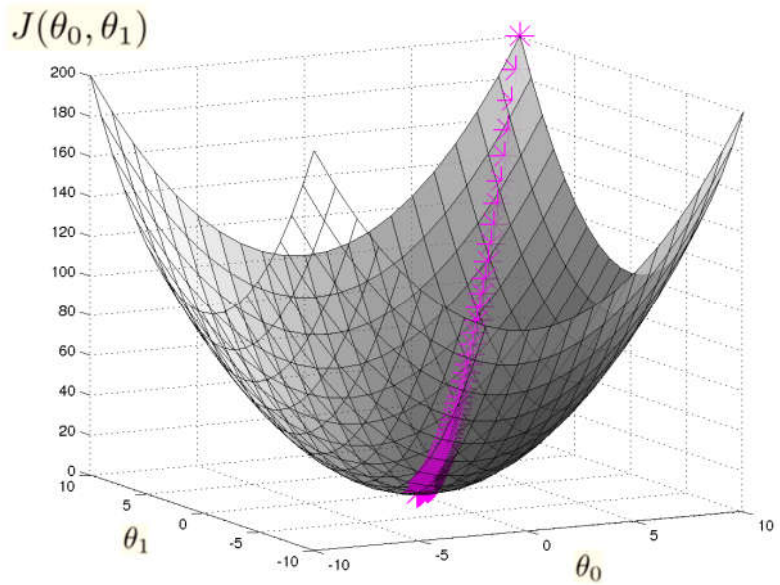
$$\underset{\theta_0,\theta_1}{\text{minimize}} \frac{1}{2m} \sum_{i=1}^{m} \left(h\left(x^i\right) - y^i\right)^2$$

- The above cost function is referred to as mean square error (MSE) function

Mean Square Error Cost Function

- $J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h\left(x^i\right) - y^i \right)^2$

Goal is to $\underset{\theta_0, \theta_1}{\text{minimize}}\, J(\theta_0, \theta_1)$



Gradient Descent Algorithm

- Initialization

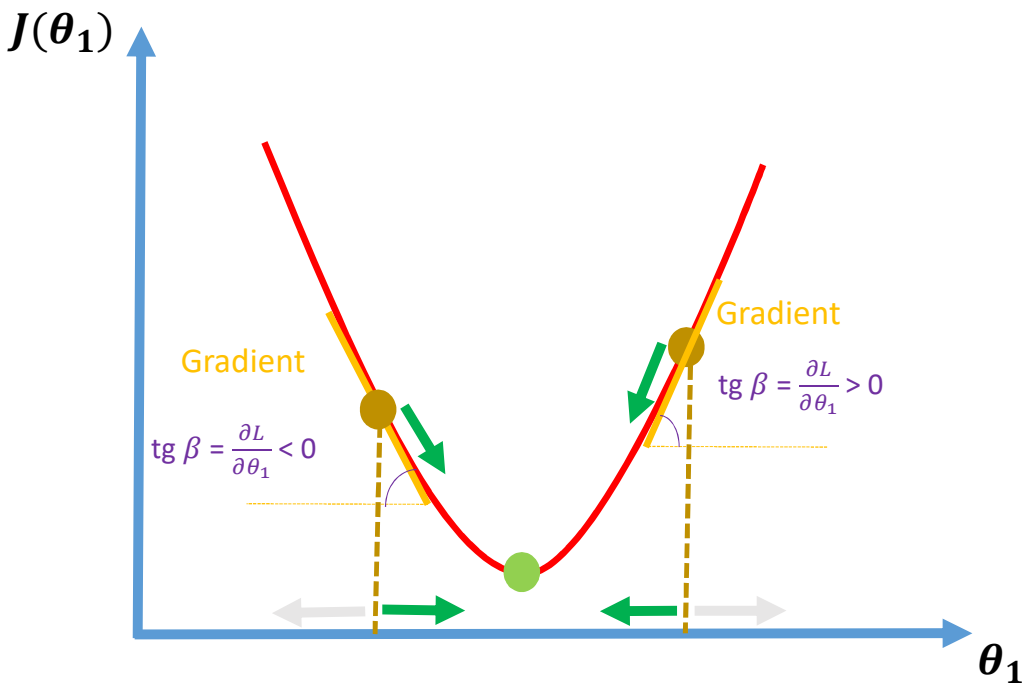
- Repeat until convergence:

for j=1 to n do

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_{0..n})$$

(subject to: $0 \leq j \leq n,\ \alpha > 0$)

Gradient Descent Algorithm (cont.)



Gradient Descent Algorithm (cont.)

- Initialization
- Repeat until convergence:

$$\boldsymbol{\theta}^{k+1} \leftarrow \boldsymbol{\theta}^{k} - \alpha \boldsymbol{\nabla}_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$$

Where gradient vector is defined as follows:

$$\boldsymbol{\nabla}_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \begin{bmatrix} \frac{\partial}{\partial \theta_0} J(\boldsymbol{\theta}) \\ \frac{\partial}{\partial \theta_1} J(\boldsymbol{\theta}) \\ \vdots \\ \frac{\partial}{\partial \theta_n} J(\boldsymbol{\theta}) \end{bmatrix}$$

Evaluation

- MSE (Mean Square Error) → RMSE (Root MSE)

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m}\left(h\left(x^i\right) - y^i\right)^2$$

- MAE (Mean Absolute Error)

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m}\left|h\left(x^i\right) - y^i\right|$$

- Which one is less sensitive to outliers? MSE or MAE?

Source

- D. Lay, J. McDonald, S. Lay, Linear Algebra and Its Applications, Chapter 6, 2020

- Andrew Ng, Gradient Descent For Linear Regression, Coursera